

Guia de Utilização de Ambiente Computacional

Usuários, Armazenamento e Contêineres Apptainer

Yan da Campo

24 de julho de 2026

Conteúdo

1	Introdução	2
2	Informações Gerais	2
3	Gerenciamento de Usuários e Acessos	2
3.1	Criação de um Novo Usuário	2
3.2	Troca de Senha pelo Próprio Usuário	2
4	Mapeamento de Armazenamento (Symlink)	2
5	Orquestração de Contêineres Apptainer	3
5.1	Criando o Arquivo de Definição	3
5.2	Construção do Contêiner	3
5.3	Executando Scripts no Contêiner	4
6	Utilização do Sistema de Filas	4
6.1	Diretrizes do Sistema de Filas	4
7	Submissão de Jobs (Usuário)	4
7.1	O Script de Submissão Padrão	4
7.2	Comandos do Dia a Dia	5
8	Introdução ao Hardware	5
9	Uso Total (Modo Monopólio)	6
9.1	Script de Poder Máximo	6
10	Uso Limitado (Alocação Fracionada)	6
10.1	Script de Poder Limitado	6
10.2	Parâmetros Críticos de Limitação	7
11	Tabela Comparativa de Hardware	7
12	Diretrizes de Configuração no Bash (#SBATCH)	7
12.1	Alocação de Processamento Máximo (Modo Monopólio)	7
12.2	Alocação e Isolamento de Memória RAM	8
12.3	Habilitação da GPU via Apptainer	8
12.4	Exemplo de Processamento em Paralelo	8
12.5	Exemplo de Processamento em Série	10
13	Extras	12
13.1	Exemplos de Arquivos de Definições de Contêineres	12
14	Considerações Finais	15

1 Introdução

Este documento descreve o procedimento padrão para a utilização das máquinas de uso comum do LCN, o mapeamento de unidades de armazenamento de alta capacidade e a orquestração de ambientes virtuais containerizados via Apptainer para cálculos de alto desempenho. Além disso, este guia contém informações sobre o sistema de filas e como gerenciar *jobs*. Em hipótese alguma simplesmente copie e cole os códigos contidos neste guia. Leia, revise e corrija, pois a formatação do PDF pode carregar espaços e símbolos indesejados.

2 Informações Gerais

Cada máquina descrita nesse guia possui suas especificidades. Leia atentamente cada seção e pesquise sobre as ferramentas utilizadas em suas documentações oficiais. Em caso de dúvidas, teste escrever seus códigos (seja de construção de contêiner, ou de submissão), escreva um *prompt* adequado e pergunte ao Modelo de Linguagem Grande de sua preferência se o que você está tentando fazer está correto. Inclusive, anexe esse PDF à sua conversa com o seu assistente de inteligência artificial. Em último caso, pergunte ao administrador do sistema de gerenciamentos (que sou eu). As seções 5, 6, 7, 8, 11, 12 e suas subseções contém as principais informações para criar o seu ambiente de recursos, arquivos de submissão, etc. Procure executar e rever seus códigos na sua máquina particular, para evitar falhas de execução. O sistema de contêineres Apptainer e o sistema de filas Slurm estão em perfeito funcionamento. Caso um *job* seu falhe, é muito mais provável que o erro de código seja seu do que meu. A tabela abaixo mostra os endereços de IP de cada máquina, bem como o super usuário e senha:

Máquina	IP
LCN02	192.168.233.223
LCNBAGUAL	192.168.234.39

Tabela 1: Dados de acesso das máquinas lcn02 e lcnbagual.

Para acessar a sua máquina de dentro da rede da UFSM basta ter qualquer recurso de acesso remoto instalado em sua máquina pessoal (como o `ssh`). O acesso será feito através do endereço de IP da respectiva máquina (vide a Tabela 1), utilizando o seu nome de usuário. Por exemplo, `ssh manguza@192.168.233.233`. O recurso `sftp` pode ser utilizado para transferência de dados. Para acesso externo à rede da Universidade, é necessária a utilização do serviço de VPN, descrito no link <https://www.ufsm.br/orgaos-suplementares/cpd/servicos/vpn-virtual-private-network-ufsm>.

3 Gerenciamento de Usuários e Acessos

3.1 Criação de um Novo Usuário

Essa etapa é feita exclusivamente pelo administrador do sistema de gerenciamento das máquinas (que sou deliberadamente eu, e não você que está lendo). Se precisar criar um usuário, acuse-se e diga o nome de usuário desejado, dentro das boas práticas de programação.

3.2 Troca de Senha pelo Próprio Usuário

Por questões de segurança, o usuário deve alterar a senha provisória imediatamente após o seu primeiro acesso. Para realizar esta alteração sem necessitar de privilégios de superusuário (`root`), basta o usuário acessar o seu próprio terminal e digitar:

```
passwd
```

O sistema solicitará a senha atual (provisória) e, em seguida, a nova senha definitiva.

4 Mapeamento de Armazenamento (Symmlink)

EXCLUSIVO PARA USUÁRIOS DO LCNBAGUAL

Para que o usuário tenha acesso rápido e direto à unidade de armazenamento de alto volume, criamos um link simbólico (atalho) que aponta do diretório raiz do disco para dentro do ambiente local do usuário. O lcnbague possui um HD SATA de 8 TB, bem como um SSD de 2 TB. Como boa prática, o usuário deve, ao final de seus projetos, transferir seus dados do SSD para o HD SATA. O link simbólico foi criado de forma a fornecer praticidade na transferência, bem como um espaço pessoal de alto volume de armazenamento para cada usuário.

Desta forma, ao acessar a pasta `~/meu_sata`, o usuário estará interagindo diretamente com os arquivos do disco principal SATA.

5 Orquestração de Contêineres Apptainer

Esta seção contém exemplos práticos de desenvolvimento de arquivos de definição e compilação de contêineres que servirão como ambientes contendo os seus softwares, recursos e ferramentas desejados. Para isso, usaremos o Apptainer – um recurso de gerenciamento de ambientes no formato de contêiner –. Sua utilização é motivada pelo isolamento dos recursos individuais, sem contaminação da raiz da máquina. A documentação oficial desta ferramenta pode ser encontrada em https://apptainer.org/docs/user/main/build_a_container.html. A subseção 13.1 apresenta alguns exemplos extras e bem detalhados de construção de contêineres para Lammmps e Gromacs.

5.1 Criando o Arquivo de Definição

A infraestrutura científica é compilada a partir de um arquivo de configuração com extensão `.def`. Este arquivo contém as diretrizes do sistema base e de todas as bibliotecas necessárias. Crie um arquivo (por exemplo, `ambiente.def`) contendo as instruções:

```
Bootstrap: docker
From: ubuntu:22.04

%post
    # Comandos de atualizacao e instalacao
    apt-get update && apt-get install -y Python3 Python3-pip
    pip3 install torch numpy matplotlib
    # (Inserir scripts de compilacao adicionais aqui)

%runscript
    exec Python3 "$@"
```

Recomenda-se a criação de uma pasta para guardar e compilar todos os seus contêineres, de forma prática e organizada como, por exemplo, uma pasta chamada **meus_ambientes** dentro da pasta pessoal do usuário.

5.2 Construção do Contêiner

A compilação do contêiner empacota o sistema operacional, os drivers e as bibliotecas em um arquivo monolítico de formato `.sif`. Este passo exige privilégios de administrador (`chmod 755`). Instalações de contêineres via arquivos de definição são feitas através da seguinte sintaxe:

```
sudo apptainer build ambiente_final.sif ambiente.def
```

Algumas aplicações e ferramentas já possuem em sua documentação instalações específicas para contêineres. O DeePMD-kit, por exemplo, possui essa opção apresentada na página <https://docs.deepmodeling.com/projects/deepmd/en/latest/install/easy-install.html>. Nesta documentação, a recomendação é que se use o comando `docker pull ghcr.io/deepmodeling/deepmd-kit:2.2.8_cuda12.0_gpu`. Como estamos usando o utilitário Apptainer, nosso comando de instalação pode ser dado por:

```
apptainer pull meu_deepmd.sif \
docker://ghcr.io/deepmodeling/deepmd-kit:2.2.8_cuda12.0_gpu
```

ou ainda

```
apptainer build meu_deepmd.sif \
docker://ghcr.io/deepmodeling/deepmd-kit:2.2.8_cuda12.0_gpu
```

5.3 Executando Scripts no Contêiner

Com o contêiner gerado, qualquer usuário pode executar seus códigos científicos usufruindo da aceleração de hardware (GPU) sem interferir nos pacotes do sistema host.

Para executar um script em Python utilizando os drivers da NVIDIA (`--nv`), utilize o comando abaixo:

```
apptainer run --nv /caminho/para/ambiente_final.sif Python3 \
    meu_script_de_analise.py
```

O comando `run` intercepta automaticamente a chamada para o `Python3` (conforme estabelecido no bloco `%runscript`), executando o arquivo `meu_script_de_analise.py` dentro do ambiente isolado.

Caso seu script não faça uso da GPU, desconsidere a flag `--nv`.

6 Utilização do Sistema de Filas

6.1 Diretrizes do Sistema de Filas

Para garantir a alocação justa de recursos e evitar o travamento da máquina por excesso de processos simultâneos, as máquinas multi usuário utilizarão o **Slurm Workload Manager**. O sistema organiza uma fila de execução, garantindo acesso exclusivo ao hardware e impondo limites de tempo de simulação. Alguns itens da utilização dos ambientes devem ser obedecidos:

- O usuário pode executar comandos e processos fora do sistema de filas, mesmo enquanto outro usuário estiver com *job* em processamento. Mas você fará isso? Pense bem antes de estragar a simulação do amiguinho;
- Use boas práticas de programação;
- Otimize seus códigos: algoritmo é importante também;
- Teste seu sistema antes de rodar em qualquer máquina: se escreveu errado e já houver outro *job* na fila, eu estarei de mãos amarradas;
- O tempo máximo de execução é 72 h. Tempos menores podem ser configurados no seu *job*;
- Um usuário só poderá submeter **um único job por vez**. Enquanto o *job* submetido não finalizar, o usuário não poderá realizar novas submissões, evitando assim a criação de uma “fila pessoal”;
- Veja as especificações da máquina utilizada na subseção 10.2 para configurar corretamente o seu *job*;
- Como estamos trabalhando com contêineres, verifique se a ferramenta que você quer usar já não possui instalação apropriada na documentação oficial;
- Seja responsável e leia o guia completo.

7 Submissão de Jobs (Usuário)

Os cálculos **não** devem ser executados diretamente no terminal com `nohup` ou em *background* (`&`). Deve-se criar um *Job Script* e enviá-lo ao Slurm.

7.1 O Script de Submissão Padrão

Crie um arquivo `.sh` (ex: `roda_gmx.sh`) contendo os parâmetros de recurso e os comandos de execução. Seu arquivo de submissão deve ter permissão 755, o que quer dizer que, após criado, você deve executar no terminal o comando `chmod 755` sucedido do nome do arquivo (ex.: `chmod 755 roda_gmx.sh`). Exemplo de um job executando processos em paralelo com Apptainer:

```
#!/bin/bash
#SBATCH --job-name=gmx_paralelo
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=8
```

```
#SBATCH --time=24:00:00
#SBATCH --output=paralelo_%j.log

# Garante que a execucao ocorra na mesma pasta da submissao
cd "$SLURM_SUBMIT_DIR"

echo "Iniciando simulacoes paralelas..."

# Execucao do processo A em background (alocando 4 threads)
apptainer exec /home/nome_usuario/meus_ambientes/gromacs_cpu.sif gmx \
    mdrun -v -deffnm npt_A -nt 4 > log_A.txt 2>&1 &

echo "Simulacao concluida."
```

Uma prática extremamente recomendada é a limpeza do seu espaço ocupado após o fim dos seus projetos. Faça backups sistemáticos!

Explicação dos Parâmetros:

- `-partition`: Define a fila alvo (neste caso, a *pesquisa*).
- `-ntasks`: Número total de núcleos requisitados.
- `-time`: Limite máximo de tempo (trava de segurança). Se o job ultrapassar, será abortado. Peça apenas o necessário.
- `-output`: Arquivo onde os *prints* gerais do sistema serão salvos (%j insere o ID dinâmico do job).
- `wait`: Comando essencial para paralelismo. Instrui o Slurm a segurar o nó até que todos os processos & terminem.

7.2 Comandos do Dia a Dia

Após criar o arquivo `.sh`, o usuário administra suas simulações através dos seguintes comandos no terminal:

```
# 1. Enviar o job para a fila
sbatch roda_gmx.sh

# 2. Ver a saude geral da maquina (idle = livre; allocated = em uso)
sinfo

# 3. Ver todos os jobs na fila (R = Rodando; PD = Pendente)
squeue

# 4. Ver apenas os jobs de um usuario especifico
squeue -u nome_do_usuario

# 5. Cancelar/Derrubar um job especifico pelo seu ID
scancel 1234

# 6. Cancelar TODOS os jobs de um usuario de uma so vez
scancel -u nome_do_usuario

# 7. Acompanhar uma simulacao em tempo real lendo o log
tail -f nome_do_arquivo_de_log.txt
```

8 Introdução ao Hardware

Para dimensionar corretamente os *jobs*, é fundamental compreender a arquitetura do nó alvo. No jargão do Slurm, o termo **CPUs** refere-se sempre ao número de **Threads** (núcleos lógicos), e não aos núcleos físicos.

Considerando um nó com 8 núcleos físicos e tecnologia *hyperthreading* (2 *threads* por núcleo) — caso da máquina LCN02 —, o limite de poder computacional do servidor é de **16 CPUs** e aproximadamente **127 GB de RAM**.

9 Uso Total (Modo Monopólio)

Para simulações pesadas que exigem o máximo de performance, o usuário deve alocar 100% dos recursos da máquina. Isso blinda o hardware contra qualquer concorrência de outros processos.

9.1 Script de Poder Máximo

O script abaixo reserva todos os 16 *threads* e bloqueia a memória RAM inteira exclusivamente para a simulação:

```
#!/bin/bash
#SBATCH --job-name=max_power
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=1          # Reserva um processo principal
#SBATCH --cpus-per-task=16   # Solicitamos todas as 16 threads logicas
#SBATCH --mem=0              # O valor '0' aloca TODA a RAM disponivel
#SBATCH --time=72:00:00
#SBATCH --output=saida_max_%j.log

cd "$SLURM_SUBMIT_DIR"

echo "Iniciando calculo em Modo Monopolio..."

# A ferramenta de dinamica molecular deve ser instruida a usar as 16 threads
apptainer exec /home/nome_usuario/meus_ambientes/gromacs_cpu.sif gmx \
    mdrun -v -deffnm producao_pesada -nt 16
```

10 Uso Limitado (Alocação Fracionada)

É uma péssima prática alocar a máquina inteira para um *job* que não tem a capacidade de escalar em múltiplos núcleos. Se a simulação requer apenas uma fração do hardware, o poder computacional deve ser limitado de forma explícita.

10.1 Script de Poder Limitado

O exemplo a seguir demonstra como restringir a alocação para apenas 4 *threads* e uma quantidade limitada de RAM, permitindo que os recursos ociosos fiquem livres caso outros *jobs* compatíveis entrem na fila (se a partição não for exclusiva).

```
#!/bin/bash
#SBATCH --job-name=teste_leve
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=1          # Reserva um processo principal
#SBATCH --cpus-per-task=8    # Aloca 8 threads para o processo
#SBATCH --mem=32G            # Limita o consumo a 32 Gigabytes de RAM
#SBATCH --time=04:00:00      # Job curto (4 horas)
#SBATCH --output=saida_leve_%j.log

cd "$SLURM_SUBMIT_DIR"

echo "Iniciando calculo limitado..."

# O container executara utilizando os limites impostos
apptainer run --nv \
    ~/meu_sata/meus_ambientes/pipeline_materiais.sif Python3 \
    meu_codigo.py --num_workers 8
```

10.2 Parâmetros Críticos de Limitação

- **-ntasks=X**: Restringe o número de *threads* que o processo principal e seus subprocessos poderão acessar. Substitua **X** pelo valor numérico desejado.
- **-mem=Y**: Impõe uma barreira rígida de consumo de RAM. O sufixo **G** indica Gigabytes e o sufixo **M** indica Megabytes (ex: **-mem=16G** ou **-mem=16000M**). Se o script tentar ultrapassar este valor, o Slurm abortará o processo automaticamente por falta de memória (*Out Of Memory - OOM*).

11 Tabela Comparativa de Hardware

A tabela seguinte sintetiza o poder computacional disponível em ambos os servidores do laboratório. Os valores de CPU e memória foram mapeados em estrita conformidade com os dados reportados pelo daemon do Slurm, servindo como métrica exata para a elaboração dos scripts de submissão. **Cada máquina tem 2 threads por núcleo. Dessa forma, em hipótese alguma solicite números ímpares de tasks no script de submissão.**

AVISO: A MÁQUINA LCNBAGUAL É DE USO EXCLUSIVO PARA TRABALHOS COM USO MASSIVO DE GPU!!!

Tabela 2: Especificações Técnicas: lcn02 vs lcnbagual

Especificação	Servidor: lcn02	Servidor: lcnbagual
Processador (Arquitetura)	1 Soquete / 1 Placa	1 Soquete / 1 Placa
Núcleos Físicos (<i>Cores</i>)	8 Núcleos	64 Núcleos
Threads (Slurm CPUs)	16 Threads (Máximo)	128 Threads (Máximo)
Memória RAM Total (Bruta)	~128 GB (128.637 MB)	~128 GB (128.133 MB)
Memória RAM Segura (Slurm)	126.000 MB	126.000 MB
Placa de Vídeo (GPU)	1x NVIDIA GeForce RTX 3060	1x NVIDIA GeForce RTX 5090

12 Diretrizes de Configuração no Bash (#SBATCH)

Para otimizar o tempo de fila e garantir estabilidade operacional, os utilizadores devem transcrever os dados da tabela diretamente para as flags de alocação do script `.sh`.

12.1 Alocação de Processamento Máximo (Modo Monopólio)

Caso a simulação consiga escalar eficientemente em paralelo, utilize o teto lógico de cada máquina na diretiva **-ntasks**:

- **No servidor lcn02:**

$\text{ntasks} \times \text{cpus-per-task} = 16$

Exemplo para 1 processo:

```
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
```

- **No servidor lcnbagual:**

$\text{ntasks} \times \text{cpus-per-task} = 128$

Exemplo para 2 processos em paralelo:

```
#SBATCH --ntasks=2
#SBATCH --cpus-per-task=64
```

12.2 Alocação e Isolamento de Memória RAM

O comando ideal para blindar o nó e requisitar toda a memória RAM disponível de forma automática, independente do servidor em execução, é definir o parâmetro como zero:

```
#SBATCH --mem=0
```

Se o objetivo for submeter um *job* fracionado leve (permitindo que outras tarefas rodem simultaneamente), defina um limite explícito respeitando a folga de segurança do sistema operativo (ex: alocar metade da capacidade total):

```
#SBATCH --mem=63G
```

12.3 Habilitação da GPU via Apptainer

As placas de vídeo (RTX 3060 e RTX 5090) são injetadas de forma transparente dentro do ecossistema containerizado. Para mapear o hardware físico para dentro do ambiente de cálculo (ativando o suporte CUDA no PyTorch ou LAMMPS), basta adicionar a flag `-nv` na chamada do contêiner:

```
apptainer run --nv \
~/meu_sata/meus_ambientes/pipeline_materiais.sif Python3 meu_script.py
```

12.4 Exemplo de Processamento em Paralelo

Caso o usuário queira executar múltiplas execuções, pode fazer isso em paralelo, gerenciando o numero de CPUs em cada processo. É muito importante que a sintaxe correta seja obedecida, seguindo o a seguinte lógica básica:

- **Para o Gromacs:** A divisão dos motores (`-n1 + -nt X`): Dado um número solicitado de tarefas no `#SBATCH`, o Slurm entrega X núcleos virtuais ao *job*. Para um dado exemplo de comando do Slurm `srun --exclusive -N1 -n1 --cpus-per-task=8` o operador cria **1 tarefa** (`-n1`) e disponibiliza **8 CPUs para esta tarefa** (`--cpus-per-task=8`). A flag `-nt 8` sinaliza ao Gromacs para que use as 8 threads reservadas. Se você não usar a flag `-nt X`, o Gromacs tentará automaticamente usar os núcleos de todos os processos simultaneamente (mesmo limitando-se ao número reservado), causando um estrangulamento de processos que tornará a simulação mais lenta do que se rodasse em série;
- **Para o Lammmps (OpenMP):** A divisão de motores (`-n1 + -sf -omp -pk omp Y`): Se a sua versão do Lammmps não tiver suporte MPI ou se desejar executar seus processos sem essa funcionalidade, deve-se proceder da seguinte maneira: para um dado comando do Slurm `srun --exclusive -N1 -n1 --cpus-per-task=8`, o operador cria **1 única tarefa** (para o caso sem MPI, deve-se usar sempre `-n1`) e disponibiliza **8 CPUs para cada tarefa** (`--cpus-per-task=8`), o que totaliza $1 \times 8 = 8$ CPUs totais isoladas para uma determinada simulação. O comando `-sf omp` diz ao Lammmps usar o pacote *Suffix OpenMP*, ativando o suporte a threads aceleradas para os cálculos. Já o comando `-pk omp 8` diz ao Lammmps para usar as 8 threads que foram reservadas para o processo;
- **Para o Lammmps (OpenMP + MPI):** A divisão de motores (`-nX + -sf omp -pk omp Y/X`): Para um dado exemplo de comando do Slurm `srun --exclusive -N1 -n2 --cpus-per-task=4`, o operador cria **2 tarefas MPI** (`-n2`) e disponibiliza **4 CPUs para cada tarefa** (`--cpus-per-task=4`). Multiplicando um pelo outro: $2 \times 4 = 8$ CPUs totais isoladas para uma determinada simulação. O comando `-sf omp` diz ao Lammmps usar o pacote *Suffix OpenMP*, ativando o suporte a threads aceleradas para os cálculos. Já o comando `-pk omp 4` diz ao Lammmps para usar as 4 threads que foram reservadas para o processo;
- **O Isolamento dos logs** (`> producao_X.log 2>&1`): Redireciona tudo o que cada processo imprimiria no terminal (incluindo os avisos de erro) para um arquivo de texto limpo. O arquivo `saida_geral_%j.log` definido no cabeçalho guardará apenas os textos do *echo*;
- **O despacho paralelo** (`&`): Colocar o `&` no final das linhas de comando instrui o bash a iniciar o Sistema N, soltá-lo em segundo plano, e passar imediatamente para a linha de baixo para ligar o Sistema N+1;

- **A trava de segurança (*wait*):** Sem o *wait*, o *bash* despacha todos os comandos e, sem mais nada para ler no arquivo, declara o *job* como "concluído" em um segundo de execução. O *wait* impõe a ordem para que o script congele exatamente ali até que todas as execuções cheguem ao fim.

Atenção: A soma das tarefas (*-n*) e CPUs (*--cpus-per-task*) solicitadas simultaneamente nos comandos *srunch* nunca pode ultrapassar o limite total definido no cabeçalho *#SBATCH* do script, incluindo a alocação por execução interna, para o caso de simulações em paralelo.

Abaixo, um exemplo de aplicação para o Gromacs (OpenMP):

```
#!/bin/bash
#SBATCH --job-name=paralelo_max
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=2          # 2 processos principais do script
#SBATCH --cpus-per-task=8    # Aloca todas as 16 threads do no para o job
#SBATCH --mem=0              # Bloqueia TODA a RAM para este job. Exclua essa linha
                             # para gerenciamento interno de memoria
#SBATCH --time=72:00:00
#SBATCH --output=saida_geral_%j.log

cd "$SLURM_SUBMIT_DIR"

echo "Iniciando calculo em paralelo: 2 sistemas dividindo o no..."

# SISTEMA 1:
# srun: Ponteiro para que o operador gerencie o processo
# --exclusive: Garante que os recursos alocados para este passo especifico do
#               job sejam de uso unico e exclusivo dele
# -N1: Define o numero de nos que este passo vai usar
# -n: Define a quantidade de tarefas ou processos principais que serao
#     disparados (ex.: um unico processo com X threads)
# --cpus-per-task: Determina quantas CPUs (nucleos logicos/threads) serao
#                  dedicadas exclusivamente para as tarefas definidas no -n
# Usa 8 threads (-nt 8 - comando exclusivo do Gromacs), redireciona a saida para
#               seu proprio log e vai para o background (&). Embora este comando e o
#               anterior parecam redundantes, nao sao. O --cpus-per-task=8 avisa ao operador
#               quantos threads reservar e o -nt injeta nas configuracoes do software o
#               numero de threads desejado, dentro do que foi reservado para o processo.

srun --exclusive -N1 -n1 --cpus-per-task=8 apptainer exec \
    /home/nome_usuario/meus_ambientes/gromacs_cpu.sif gmx \
    mdrun -v -deffnm producao_pesada_1 -nt 8 > producao_1.log 2>&1 &

# SISTEMA 2:
# Usa os 8 threads restantes (-nt 8), tem seu proprio log e vai para o
#               background (&)

srun --exclusive -N1 -n1 --cpus-per-task=8 apptainer exec \
    /home/nome_usuario/meus_ambientes/gromacs_cpu.sif gmx \
    mdrun -v -deffnm producao_pesada_2 -nt 8 > producao_2.log 2>&1 &

echo "Simulacoes em andamento. O no ficara bloqueado ate a conclusao..."

# O comando magico de sincronizacao:
# Faz o Slurm esperar ate que as tarefas 1 e 2 terminem antes de encerrar o job
wait

echo "Todas as simulacoes foram concluidas com sucesso!"
```

Exemplo LAMMPS (OpenMP + MPI):

```
#!/bin/bash
#SBATCH --job-name=lammps_hibrido
#SBATCH --partition=pesquisa
```

```

#SBATCH --nodes=1
#SBATCH --ntasks=4                # 4 processos MPI totais (2 para Sim A + 2
    para Sim B)
#SBATCH --cpus-per-task=4         # 4 threads OpenMP por processo MPI
#SBATCH --time=30:00:00
#SBATCH --output=lammps_%j.log

cd "$SLURM_SUBMIT_DIR"
echo "Iniciando simulacoes paralelas de LAMMPS (MPI + OpenMP)..."

# --- SIMULACAO A (2 tarefas MPI x 4 threads OpenMP = 8 threads) ---
srun --exclusive -N1 -n2 --cpus-per-task=4 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 4 -in in.sistemaA > log_simulacaoA.txt 2>&1 &

# --- SIMULACAO B (2 tarefas MPI x 4 threads OpenMP = 8 threads) ---
srun --exclusive -N1 -n2 --cpus-per-task=4 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 4 -in in.sistemaB > log_simulacaoB.txt 2>&1 &

# Impede que o script feche antes das simulacoes terminarem
wait

echo "Simulacoes do LAMMPS concluidas."

```

Exemplo Lammps (OpenMP):

```

#!/bin/bash
#SBATCH --job-name=lammps_omp_puro
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=2                # 2 processos principais (1 para Sim A, 1
    para Sim B)
#SBATCH --cpus-per-task=8         # 8 threads OpenMP por processo
#SBATCH --time=30:00:00
#SBATCH --output=lammps_%j.log

cd "$SLURM_SUBMIT_DIR"
echo "Iniciando simulacoes paralelas de LAMMPS (Apenas OpenMP)..."

# --- SIMULACAO A (1 tarefa x 8 threads) ---
srun --exclusive -N1 -n1 --cpus-per-task=8 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 8 -in in.sistemaA > log_simulacaoA.txt 2>&1 &

# --- SIMULACAO B (1 tarefa x 8 threads) ---
srun --exclusive -N1 -n1 --cpus-per-task=8 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 8 -in in.sistemaB > log_simulacaoB.txt 2>&1 &

# Impede que o script feche antes das simulacoes terminarem
wait

echo "Simulacoes do LAMMPS concluidas."

```

12.5 Exemplo de Processamento em Série

Abaixo, dois exemplos de processos em série. Perceba que não usamos o gerenciamento do operador Slurm, o despacho paralelo (&) e nem a trava *wait*.

Exemplo Gromacs:

```

#!/bin/bash
#SBATCH --job-name=gmx_serial

```

```

#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16          # Aloca todas as 16 threads logicas
#SBATCH --time=30:00:00
#SBATCH --output=serial_%j.log

cd "$SLURM_SUBMIT_DIR"
echo "Iniciando simulacoes sequenciais em capacidade maxima..."

# Simulacao 1 usa todas as 16 threads do no
echo "Rodando simulacao 1..."
apptainer exec /home/nome_usuario/meus_ambientes/gromacs_cpu.sif \
gmx mdrun -v -deffnm producao_pesada_1 -nt 16 > producao_1.log 2>&1

# Simulacao 2 assume o no inteiro logo em seguida
echo "Rodando simulacao 2..."
apptainer exec /home/nome_usuario/meus_ambientes/gromacs_cpu.sif \
gmx mdrun -v -deffnm producao_pesada_2 -nt 16 > producao_2.log 2>&1

echo "Simulacoes concluidas."

```

Exemplo Lammmps (OpenMP + MPI):

```

#!/bin/bash
#SBATCH --job-name=lammmps_serial_hibrido
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=4          # CORRECAO: 4 processos MPI autorizados
#SBATCH --cpus-per-task=4    # CORRECAO: 4 threads OpenMP por processo
#SBATCH --time=30:00:00
#SBATCH --output=lammmps_%j.log

cd "$SLURM_SUBMIT_DIR"
echo "Iniciando simulacoes sequenciais de LAMMPS (Hibrido: 4 MPI x 4 OMP)..."

# --- SIMULACAO A ---
echo "Rodando simulacao A..."
srun -n4 --cpus-per-task=4 \
apptainer exec /home/nome_usuario/meus_ambientes/lammmps_cpu.sif \
lmp -sf omp -pk omp 4 -in in.sistemaA > log_simulacaoA.txt 2>&1

# Opcional, mas altamente recomendado devido ao historico do seu NVMe
echo "Pausa para resfriamento do SSD..."
sleep 120

# --- SIMULACAO B ---
echo "Rodando simulacao B..."
srun -n4 --cpus-per-task=4 \
apptainer exec /home/nome_usuario/meus_ambientes/lammmps_cpu.sif \
lmp -sf omp -pk omp 4 -in in.sistemaB > log_simulacaoB.txt 2>&1

echo "Todas as simulacoes do LAMMPS foram concluidas."

```

Exemplo Lammmps (OpenMP):

```

#!/bin/bash
#SBATCH --job-name=lammmps_serial_omp
#SBATCH --partition=pesquisa
#SBATCH --nodes=1
#SBATCH --ntasks=1          # 1 unico processo principal
#SBATCH --cpus-per-task=16   # 16 threads alocadas para este processo
#SBATCH --time=30:00:00
#SBATCH --output=lammmps_%j.log

```

```

cd "$SLURM_SUBMIT_DIR"
echo "Iniciando simulacoes sequenciais de LAMMPS (OpenMP Puro: 16 threads)..."

# --- SIMULACAO A ---
echo "Rodando simulacao A..."
srun -n1 --cpus-per-task=16 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 16 -in in.sistemaA > log_simulacaoA.txt 2>&1

# Opcional, mas altamente recomendado devido ao historico do seu NVMe
echo "Pausa para resfriamento do SSD..."
sleep 120

# --- SIMULACAO B ---
echo "Rodando simulacao B..."
srun -n1 --cpus-per-task=16 \
apptainer exec /home/nome_usuario/meus_ambientes/lammps_cpu.sif \
lmp -sf omp -pk omp 16 -in in.sistemaB > log_simulacaoB.txt 2>&1

echo "Todas as simulacoes do LAMMPS foram concluidas."

```

13 Extras

13.1 Exemplos de Arquivos de Definições de Contêineres

Instalação de versão atualizada do Lammps (OpenMP + MPI) + Python:

```

Bootstrap: docker
From: ubuntu:22.04

%environment
export LC_ALL=C
export DEBIAN_FRONTEND=noninteractive
export PythonUNBUFFERED=1
# Define o OpenMP para usar threads disponiveis (ajuste conforme necessario)
export OMP_NUM_THREADS=1

%post
export DEBIAN_FRONTEND=noninteractive

echo "=== 1. Atualizando sistema base ==="
apt-get update && apt-get upgrade -y
apt-get install -y --no-install-recommends \
    build-essential \
    git \
    cmake \
    wget \
    curl \
    Python3 \
    Python3-pip \
    Python3-dev \
    openmpi-bin \
    libopenmpi-dev \
    libfftw3-dev

echo "=== 2. Clonando o LAMMPS ==="
cd /opt
git clone --depth 1 -b stable https://github.com/lammps/lammps.git
    lammps_src

echo "=== 3. Compilando o LAMMPS (Somente CPU + MPI + OpenMP) ==="
mkdir -p /opt/lammps_src/build

```

```

cd /opt/lammps_src/build

# Configuracao do CMake focada exclusivamente em CPU de alta performance
cmake ../cmake \
  -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_MPI=yes \
  -DBUILD_SHARED_LIBS=yes \
  -DCMAKE_CXX_STANDARD=17 \
  -DPKG_OPENMP=yes \
  -DPKG_Python=yes \
  -DPKG_MANYBODY=yes \
  -DPKG_MOLECULE=yes \
  -DPKG_KSPACE=yes \
  -DFFT3_MKL=no

# Compila usando 16 nucleos paralelamente dentro do container
make -j 2
make install
make install-Python

echo "=== Ambiente LAMMPS CPU Compilado com Sucesso! ==="

```

Instalação atualizada do Gromacs (OpenMP) + Python + bibliotecas de análises de dados e matemática:

```

BootStrap: docker
From: ubuntu:22.04

%help
  Container Apptainer contendo o GROMACS 2023.3 (Versao CPU)
  Ignorando os testes de regressao estritos para compatibilidade com
  arquitetura hibrida.

%environment
  export PATH="/usr/local/gromacs/bin:$PATH"
  export LD_LIBRARY_PATH="/usr/local/gromacs/lib64:$LD_LIBRARY_PATH"
  export PKG_CONFIG_PATH="/usr/local/gromacs/lib64/pkgconfig:$PKG_CONFIG_PATH"
  export LC_ALL=C.UTF-8
  export LANG=C.UTF-8

%post
  export DEBIAN_FRONTEND=noninteractive
  apt-get update && apt-get upgrade -y
  apt-get install -y \
    build-essential \
    gcc \
    g++ \
    cmake \
    wget \
    zlib1g-dev \
    libfftw3-dev \
    Python3 \
    Python3-pip \
    Python3-dev

  pip3 install --no-cache-dir numpy pandas matplotlib

  rm -rf /tmp/gromacs-build
  mkdir -p /tmp/gromacs-build
  cd /tmp/gromacs-build

  wget https://ftp.gromacs.org/gromacs/gromacs-2023.3.tar.gz
  # Nao precisamos mais baixar os testes de regressao se nao vamos roda-los

```

```

tar -xzf gromacs-2023.3.tar.gz

cd gromacs-2023.3
rm -rf build
mkdir -p build
cd build

cmake .. \
  -DCMAKE_INSTALL_PREFIX=/usr/local/gromacs \
  -DGMX_BUILD_OWN_FFTW=OFF \
  -DGMX_GPU=OFF \
  -DGMX_USE_OPENCL=OFF \
  -DGMX_SIMD=AVX2_256

THREADS=4

# Compilacao principal
make -j 2

# COMENTADO: Ignoramos os testes que travam o pipeline por falsos-positivos
#             numericos
# make check -j 2

# Instalacao direta do binario compilado
make install

cd /
rm -rf /tmp/gromacs-build
apt-get clean
rm -rf /var/lib/apt/lists/*

%runscript
  exec gmx "$@"

%test
  gmx --version
  Python3 -c "import numpy; import pandas; import matplotlib; print('Ambiente
    Python: OK!')"

```

Instalação de Python + PyTorch + CUDA:

```

BootStrap: docker
# Certifique-se da versao do cuda aceita pela maquina de interesse e a versao
# compativel com as ferramentas que ira utilizar
From : nvidia / cuda :12.6 - cudnn - devel - ubuntu22 .04
%post
  export DEBIAN_FRONTEND=noninteractive
  apt-get update && apt-get upgrade -y
  apt-get install -y \
    build-essential \
    gcc \
    g++ \
    zlib1g-dev \
    libfftw3-dev \
    Python3 \
    Python3-pip \
    Python3-dev

  pip3 install --no-cache-dir numpy pandas matplotlib

  pip3 install torch torchvision --index-url https://download.pytorch.org/whl/
    cu126

%runscript

```

```
exec Python3 "$@"
```

14 Considerações Finais

Leia atentamente o guia, teste seus scripts em suas máquinas pessoais antes de submetê-los, use o recurso de Modelo de Linguagem Grande de sua preferência a seu favor e, em último caso, me consulte. Eu eventualmente estarei de olho nas execuções de cada máquina, então não tente nenhuma gracinha. Use boas práticas de programação, otimize seus scripts, evite submeter muitos *jobs* em sequência e esvazie seus ambientes após rodadas de simulação, pois nosso recurso de armazenamento é limitado e coletivo, e eu também estarei de olho em cada byte que vocês ocuparem. Não conseguiu acessar o PC via ssh? Olhe pela janela da 1036 e veja se as máquinas estão ligadas. As máquinas estão ligadas? Contate o CPD!